# CS1312
# Algorithms Design and Analysis

## Fundamentals of Algorithms Analysis

# What is algorithm analysis?

- Analysis of algorithms refers to the task of determining how "good" an algorithm is, usually in terms of running time and storage.
  - It allows you to make quantitative judgments about the value of one algorithm over another.
  - It allows you to predict whether the software will meet any efficiency constraints that exist.

# What is algorithm analysis?

- Questions such as how well does an algorithm perform in the:
  - best, worst and average cases.

# What is algorithm analysis?

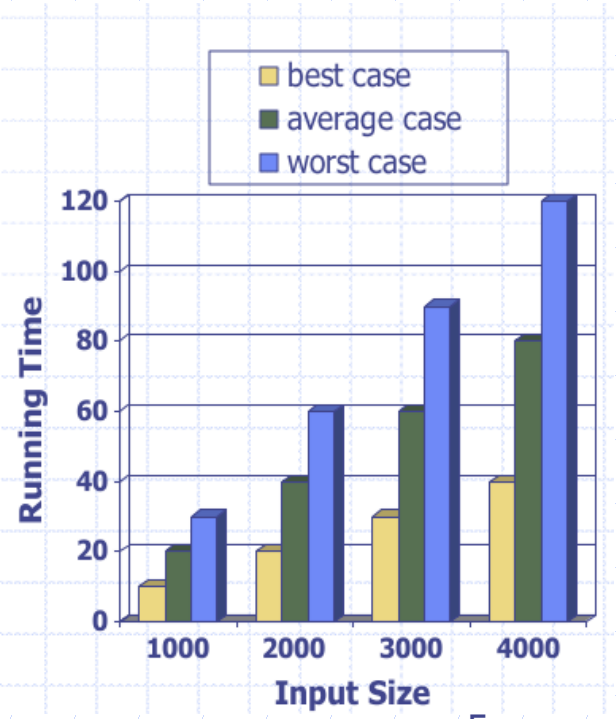
- If computers were infinitely fast and computer memory was free, you would still like to demonstrate that your algorithm terminates and does so with the correct answer.

- Computers may be fast, but they are not infinitely fast. And memory may be inexpensive, but it is not free

  ➔ running time and memory are bounded resources

# What is running time?

- The running time of an algorithm typically grows with the input size.
- How long an algorithm needs to produce the output.
- Three estimated running time:
  - best,
  - average and
  - worst



# How to analyze an algorithm?

- The main tool is to measure the running time following one of these approaches:

| | |
|---|---|
| **Experimental Study** | • A real program.<br>• Some typical inputs.<br>• Depends on hardware/ software implementation. |
| **Theoretical analysis** | • All possible inputs.<br>• Independent of hardware/ software implementation. |

# Experimental Study

- Write a program implementing the algorithm.
- Run the program with different inputs.
- Use a function, like the built-in clock() function, to get an accurate measure of the actual running time.
- Plot the results.

# Experimental Study

- **Limitations** :
  - It is necessary to implement the algorithm.
  - Results may not be indicative of the running time on other inputs not included in the experiment.
  - In order to compare two algorithms, the same hardware and software environments must be used.

# Theoretical Analysis

- Uses a high-level description of the algorithm instead of an implementation.
- Represents running time based on the number of primitive operations or steps executed.
- Characterizes running time as a function of the input size, n.
- Takes into account all possible input sizes.
- Allows evaluating the algorithm speed independently of the HW/SW environment.

# Theoretical Analysis (cont.)

- Primitive operations are the basic computations performed by an algorithm.

- Examples:
  - Evaluating an expression
  - Assigning a value to a variable
  - Indexing into an array
  - Calling a method
  - Returning from a method

# Theoretical Analysis (cont.)

- **Example1:** Loop

```
i=1
loop (i<=1000)
  application code
  i=i+1
end loop
```

f(n)=1000

| iteration | Value of i |
|---|---|
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |
| … | … |
| 1000 | 1000 |

1000

# Theoretical Analysis (cont.)

- **Example1:** Linear loop

```
i=1
loop (i<=n)
  application code
  i=i+1
end loop
```

| iteration | Value of i |
|---|---|
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |
| … | … |
| 1000 | 1000 |

n=1000

f(n)=n

- What if we change the incremental condition? Ex: i=i x 2

# Theoretical Analysis (cont.)

- **Example2:** Logarithmic loop (multiply)

```
i=1
loop (i<n)
  application code
  i=i x 2
end loop
```

$f(n) = \log_2 n$

| iteration | Value of i | Value of i |
|---|---|---|
| 1 | 1 | 1 |
| 2 | $2^1$ | 2 |
| 3 | $2^2$ | 4 |
| 4 | $2^3$ | 8 |
| 5 | $2^4$ | 16 |
| 6 | $2^5$ | 32 |
| 7 | $2^6$ | 64 |
| 8 | $2^7$ | 128 |
| 9 | $2^8$ | 256 |
| 10 | $2^9$ | 512 |
| Exit | $2^{10}$ | 1024 |

if n=1000
$2^{\text{iteration}} < n$

# Theoretical Analysis (cont.)

- **Example3:** Logarithmic loop (divide)

```
i=n
loop (i>=1)
  application code
  i=i / 2
end loop
```

$f(n)= \log_2 n$

| iteration | | Value of i |
|---|---|---|
| 1 | 1000 | 1000 |
| 2 | $1000/2^{1}$ | 500 |
| 3 | $1000/2^{2}$ | 250 |
| 4 | $1000/2^{3}$ | 125 |
| 5 | $1000/2^{4}$ | 62 |
| 6 | $1000/2^{5}$ | 31 |
| 7 | $1000/2^{6}$ | 15 |
| 8 | $10000/2^{7}$ | 7 |
| 9 | $1000/2^{8}$ | 3 |
| 10 | $1000/2^{9}$ | 1 |
| Exit | $1000/2^{10}$ | 0 |

If n=1000
$n / 2^{\text{iteration}} >= 1$

# Theoretical Analysis (cont.)

- **Example3:** Logarithmic loop (divide)

```
i=n
loop (i>=1)
  application code
  i=i / 10
end loop
```

| iteration | | Value of i |
|---|---|---|
| 1 | 1000 | 1000 |
| 2 | $1000/10^{1}$ | 100 |
| 3 | $1000/10^{2}$ | 10 |
| 4 | $1000/10^{3}$ | 1 |
| Exit | $1000/10^{4}$ | 0 |

If n=1000
$n / 10^{\text{iteration}} >= 1$

$f(n) = \log_{10} n$

# Theoretical Analysis (cont.)

- **Example4:** Nested loop (Quadratic )
  - **Iterations = inner loop iterations x outer loop iterations**

```
i=1
loop (i<=n)
  j=1
  loop (j<=n)
    application code
    j= j +1
  end loop
  i=i + 1
end loop
```

n

n

$f(n)= n^2$

# Theoretical Analysis (cont.)

- **Example5:** Nested loop

```
i=1
loop (i<=n)
  j=1
  loop (j<=n)
    application code
    j= j *2
  end loop
  i=i + 1
end loop
```

n

log n

f(n)= n log n

# Theoretical Analysis (cont.)

- **Example5:**

```
for (i = 0; i < n; i++)
        sum++;
for (j = 1; j <= n; j*=2)
        sum++;
```

n

log n

f(n)= n + log n

# Theoretical Analysis (cont.)

- **Example6:** Nested loop (Dependent Quadratic )

```
i=1
loop (i<=n)
  j=1
  loop (j<=i)
    application code
    j= j + 1
  end loop
  i=i + 1
end loop
```

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

If n=10

1 + 2 + 3 + … + 9 + 10 = 55

the average is 55/10 =5.5

(n+1) / 2

f(n)=n [(n+1)/2]

# Theoretical Analysis (cont.)

- **Example6:** Nested loop

```
for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            if (a[i][j] == x) return 1 ;
return -1;
```

n

m

f(n)=n.m

# Theoretical Analysis (cont.)

- **Example6:** Nested loop

```
sum = 0;
for( i = 0; i < n; i++)
        for( j = 0; j < n * n; j++)
                sum++;
```

n

$n^2$

$f(n)= n^3$

# Theoretical Analysis (cont.)

- **Example6:** if statement. The running time is the maximum of times S1 and S2.
- If S1 runs n times and S2 runs $n^2$ times, then the maximum is $n^2$.

```
if C
        s1;
else
        s2;
```

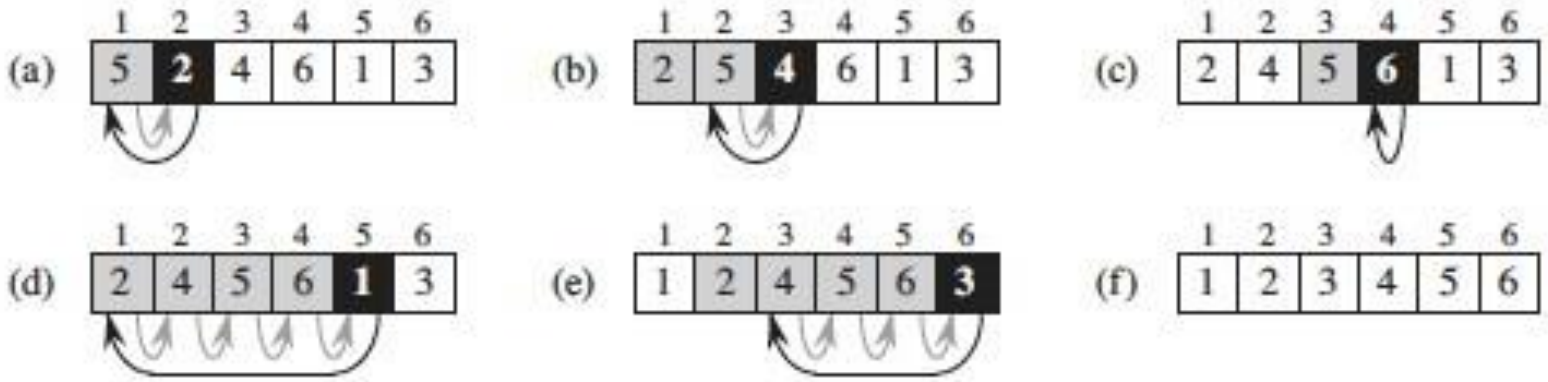
# Theoretical Analysis (cont.)

- **Example7:** Insertion sort
  - Input : sequence ⟨a1, a2, …, an⟩ of numbers.
  - Output: permutation ⟨a'1, a'2, …, a'n⟩ such that a'1≤a'2≤…≤a'n.

Example:

- Input:8 2 4 9 3 6
- Output:2 3 4 6 8 9

# Theoretical Analysis (cont.)



# Theoretical Analysis (cont.)

```
INSERTION-SORT(A)
1   for j = 2 to A.length
2       key = A[j]
3       // Insert A[j] into the sorted
            sequence A[1 .. j − 1].
4       i = j − 1
5       while i > 0 and A[i] > key
6           A[i + 1] = A[i]
7           i = i − 1
8       A[i + 1] = key
```

# Theoretical Analysis (cont.)

To counting primitive operations:

Determine the maximum number of primitive operations executed by the algorithm, as a function of the input size.

| INSERTION-SORT$(A)$ | *cost* | *times* |
|---|---|---|
| 1 **for** $j = 2$ **to** $A.length$ | $c_1$ | $n$ |
| 2 $key = A[j]$ | $c_2$ | $n-1$ |
| 3 // Insert $A[j]$ into the sorted sequence $A[1..j-1]$. | 0 | $n-1$ |
| 4 $i = j-1$ | $c_4$ | $n-1$ |
| 5 **while** $i > 0$ and $A[i] > key$ | $c_5$ | $\sum_{j=2}^{n} t_j$ |
| 6 $A[i+1] = A[i]$ | $c_6$ | $\sum_{j=2}^{n} (t_j - 1)$ |
| 7 $i = i-1$ | $c_7$ | $\sum_{j=2}^{n} (t_j - 1)$ |
| 8 $A[i+1] = key$ | $c_8$ | $n-1$ |

# Theoretical Analysis (cont.)

To counting primitive operations:

$$\sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1 \qquad \sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}$$

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\ &\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n \\ &\quad - (c_2 + c_4 + c_5 + c_8)\,. \end{aligned}$$

# Theoretical Analysis (cont.)

- Notes:
  - Algorithm Insertion-Sort executes $an^2+bn+c$ primitive operations.
  - The quadratic growth rate of the running time T(n) is an intrinsic property of algorithm Insertion-Sort.
  - Changing the hardware/software environment will affects by a constant factor but does not alter the growth rate of T(n).